MICROSERVICES PRACTICAL PROJECT

Detailed Question Paper

Project Title: Online Learning Management System

Technology: Java, Spring Boot, Spring Cloud, Eureka Server, Eureka Client, OpenFeign, REST API and MySQL

Project Structure: 1 Eureka Server + 7 Business Microservices

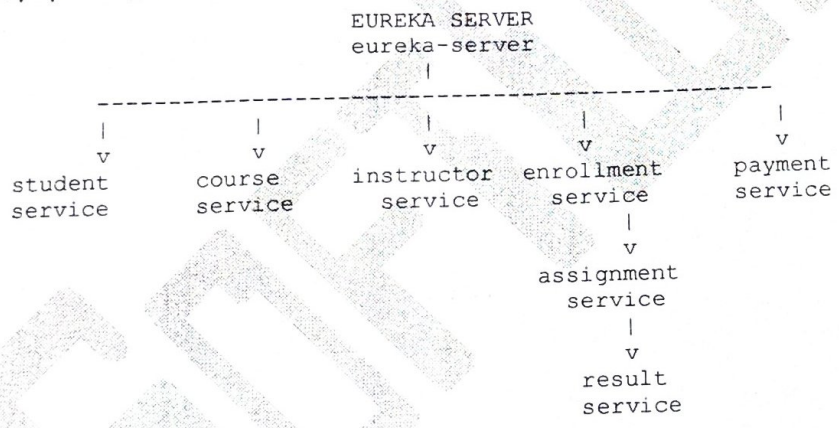
Objective: Build a complete microservices application in which each business responsibility is separated into an independent service. All services must register with Eureka, and specified inter-service communication must use OpenFeign with Eureka Service Discovery.

Important: This is an implementation-oriented project question. The names, responsibilities and communication relationships are fixed below. The student is responsible for designing the classes, database tables, controllers, services, repositories, DTOs, Feign clients, configurations and exception handling

1. Project Overview

An online learning platform must manage students, courses, instructors, enrollments, payments, assignments and results. Instead of one large monolithic application, divide the system into seven independent business microservices.

Each service owns its responsibility and business data. A service must obtain another service's information through REST/OpenFeign rather than directly accessing another service's database.



2. Basic Terms Used in This Project

Microservices Architecture
The application is divided into small independent services; each service has a specific business responsibility and runs as a separate Spring Boot application.

Spring Boot
Used to create and run each independent microservice.

Spring Cloud
Provides distributed-system tools. In this project, Eureka is used for service discovery and OpenFeign for service-to-service calls.

Eureka Server
Central service registry where microservices register themselves and from which service locations can be discovered.

Eureka Client
Each business microservice acts as a Eureka Client and registers with eureka-server.

OpenFeign
A declarative HTTP client used by one microservice to call another using the target service name.

Service Discovery
The caller asks Eureka for an available instance instead of using a fixed host and port.

3. Applications to Be Created

Type	Exact Name	Count
Infrastructure	eureka-server	1
Business	student-service	1
Business	course-service	1
Business	instructor-service	1
Business	enrollment-service	1
Business	payment-service	1
Business	assignment-service	1
Business	result-service	1

4. Student Service

Service Name: student-service

A. Purpose
Central source for student information and profile management.

B. Main Responsibilities
- Register students.
- Maintain student profiles.
- Find a student by ID.
- Return all students.
- Update and delete students.
- Provide student details to other services through REST APIs.

C. Minimum Data
```
id
name
email
mobile
```

D. Required REST Operations
- Add student
- Get student by ID
- Get all students
- Update student
- Delete student

F. Business Example / Flow
Student abhi has ID 101. When enrollment-service receives studentId=101, it calls student-service to verify and retrieve abhi's details.

5. Course Service

Service Name: course-service

A. Purpose
Manages all courses offered by the platform.

B. Main Responsibilities
- Create and maintain courses.
- Store course name, duration and fee.
- Find a course by ID.
- Return all courses.
- Update and delete courses.
- Provide course details to enrollment-service and assignment-service.

C. Minimum Data
```
id
courseName
duration
fee
```

D. Required REST Operations
- Add course
- Get course by ID
- Get all courses
- Update course
- Delete course

F. Business Example / Flow
Java course has ID 501. enrollment-service calls course-service to verify that Course 501 exists before creating an enrollment.

6. Instructor Service

Service Name: instructor-service

A. Purpose
Manages instructors who teach courses or are responsible for assignments.

B. Main Responsibilities
- Register instructors.
- Maintain instructor profiles.
- Store specialization.
- Find instructor by ID.
- Update and delete instructors.
- Provide instructor details to assignment-service.

C. Minimum Data
```
id
name
email
specialization
```

D. Required REST Operations
- Add instructor
- Get instructor by ID
- Get all instructors
- Update instructor
- Delete instructor

F. Business Example / Flow
Instructor 301 is a Java specialist. assignment-service calls instructor-service to verify instructor 301 before creating an assignment.

7. Enrollment Service

Service Name: enrollment-service

A. Purpose
Manages the relationship between a student and a course and records course enrollment.

B. Main Responsibilities
- Receive enrollment requests.
- Verify student existence.
- Verify course existence.
- Create enrollment records.
- Maintain enrollment status.
- Provide enrollment information to payment-service and result-service.

C. Minimum Data
```
id
studentId
courseId
enrollmentDate
status
```

D. Required REST Operations
- Create enrollment
- Get enrollment by ID
- Get all enrollments
- Update enrollment
- Delete enrollment

E. Required Communication
When an enrollment is created, call both student-service and course-service. Use OpenFeign for the calls and Eureka for discovery.

```
enrollment-service
        |
        +----> student-service
        |
        +----> course-service
```

Do not use fixed host/port URLs to locate these services.

F. Business Example / Flow
Student 101 wants Course 501. enrollment-service obtains Student 101 and Course 501 through the two target services. If valid, it creates the enrollment.

8. Payment Service

Service Name: payment-service

A. Purpose
Manages payments made for a student's course enrollment.

B. Main Responsibilities
- Receive payment requests.
- Verify student.
- Verify enrollment.
- Store amount, date and status.
- Provide payment details/status.

C. Minimum Data
```
id
studentId
enrollmentId
amount
```



```
paymentDate
status
```

D. Required REST Operations
- Create payment
- Get payment by ID
- Get all payments
- Update payment status
- Delete payment

E. Required Communication
Before processing payment, call student-service and enrollment-service using OpenFeign + Eureka.

```
payment-service
        |
        +----> student-service
        |
        +----> enrollment-service
```

F. Business Example / Flow
Enrollment 9001 belongs to Student 101. payment-service verifies the student and enrollment before recording the payment.

9. Assignment Service

Service Name: assignment-service

A. Purpose
Manages assignments associated with courses and instructors.

B. Main Responsibilities
- Create assignments.
- Store title and description.
- Associate assignment with course.
- Associate assignment with instructor.
- Maintain due date.
- Provide assignment information to result-service.

C. Minimum Data
```
id
title
description
courseId
instructorId
dueDate
```

D. Required REST Operations
- Create assignment
- Get assignment by ID
- Get all assignments
- Update assignment
- Delete assignment

E. Required Communication
Communicate with course-service and instructor-service using OpenFeign + Eureka.

```
assignment-service
        |
        +----> course-service
        |
        +----> instructor-service
```

F. Business Example / Flow

Before saving an assignment for Course 501 and Instructor 301, verify both through their respective services.

10. Result Service

Service Name: result-service

A. Purpose

Manages marks and grades obtained by students in assignments or examinations.

B. Main Responsibilities

- Create and maintain results.
- Associate result with student.
- Associate result with assignment.
- Associate result with enrollment.
- Store marks and grade.
- Provide result information.

C. Minimum Data

```
id
studentId
assignmentId
enrollmentId
marks
grade
resultDate
```

D. Required REST Operations

- Create result
- Get result by ID
- Get all results
- Update result
- Delete result

E. Required Communication

Communicate with student-service, assignment-service and enrollment-service using OpenFeign + Eureka.

```
result-service
      |
      +----> student-service
      +----> assignment-service
      +----> enrollment-service
```

F. Business Example / Flow

Student 101 submits Assignment 7001. result-service verifies student, assignment and enrollment before saving marks and grade.

11. Complete Service-to-Service Communication Matrix

Calling Service	Target Service	Reason	Technology
enrollment-service	student-service	Verify/retrieve student	OpenFeign + Eureka
enrollment-service	course-service	Verify/retrieve course	OpenFeign + Eureka
payment-service	student-service	Verify student	OpenFeign + Eureka
payment-service	enrollment-service	Verify enrollment	OpenFeign + Eureka
assignment-service	course-service	Verify course	OpenFeign + Eureka
assignment-service	instructor-service	Verify instructor	OpenFeign + Eureka
result-service	student-service	Verify student	OpenFeign + Eureka
result-service	assignment-service	Verify assignment	OpenFeign + Eureka
result-service	enrollment-service	Verify enrollment	OpenFeign + Eureka

12. Complete End-to-End Business Flow

Scenario: A student enrolls in a course, makes payment, receives an assignment and gets a result.

```
Step 1: Student 101 exists in student-service.
Step 2: Course 501 exists in course-service.
Step 3: enrollment-service -> student-service + course-service -> creates enrollment.
Step 4: payment-service -> student-service + course-service -> creates payment.
Step 5: assignment-service -> course-service + instructor-service -> creates assignment.
Step 6: result-service -> student-service + assignment-service + enrollment-service -> creates
result.
```

This scenario should be used as the main project demonstration.

13. Eureka Server and Registration Requirements

Create eureka-server as a separate Spring Boot application and configure it as the Eureka Server. All seven business services must act as Eureka Clients.

```
Eureka Registry
      |
      +-- student-service
      +-- course-service
      +-- instructor-service
      +-- enrollment-service
      +-- payment-service
      +-- assignment-service
      +-- result-service
```

The Eureka Dashboard must show all seven business services as registered/available.

14. OpenFeign Requirements

For every mandatory relationship in the communication matrix, create an appropriate Feign client in the calling service.

The Feign client must target the required service by its service name, while Eureka discovers an available instance.

```
Calling Service
      |
      | OpenFeign
      v
Target Service
      |
      | discovered through Eureka
      v
Available Service Instance
```

Do not hard-code another service's IP address or port as the discovery mechanism.

15. Database Requirements

Each business microservice must own its business data. Separate databases or clearly separated schemas/tables may be used according to the implementation.

- student-service owns student data.
- course-service owns course data.
- instructor-service owns instructor data.
- enrollment-service owns enrollment data.
- payment-service owns payment data.
- assignment-service owns assignment data.
- result-service owns result data.
- A service must not directly access another service's database.

16. REST API Expectations

Each service must expose meaningful REST endpoints for its own CRUD operations. Exact URL naming may be designed by the student.

```
POST   /students
GET    /students/{id}
GET    /students
PUT    /students/{id}
DELETE /students/{id}
```

Use the same CRUD concept for courses, instructors, enrollments, payments, assignments and results.

17. Validation and Business Rules

- Reject enrollment if student does not exist.
- Reject enrollment if course does not exist.
- For payment, verify referenced student and enrollment.
- For assignment, verify course and instructor.
- For result, verify student, assignment and enrollment.
- Return suitable HTTP status codes and meaningful error responses.
- Do not create records for invalid references.

18. Service Failure Consideration

Microservices are independent applications. If a required target service is unavailable, the calling service should return an understandable error rather than pretending that remote data was obtained.

Optional enhancement: implement fallback/error-handling mechanisms if covered in the course.

19. Practical Demonstration

Step 1: Start eureka-server.
Step 2: Start all seven business microservices.
Step 3: Show all seven services on the Eureka Dashboard.
Step 4: Insert sample students, courses and instructors.
Step 5: Create an enrollment and demonstrate calls to student-service and course-service.
Step 6: Create a payment and demonstrate calls to student-service and enrollment-service.
Step 7: Create an assignment and demonstrate calls to course-service and instructor-service.
Step 8: Create a result and demonstrate calls to student-service, assignment-service and enrollment-service.
Step 9: Demonstrate communication through service names/Eureka rather than fixed target ports.

20. Final Project Summary

1. eureka-server
2. student-service
3. course-service
4. instructor-service
5. enrollment-service
6. payment-service
7. assignment-service
8. result-service

Total: 8 Spring Boot applications = 1 Eureka Server + 7 business microservices.

Core rule: Eureka discovers services; OpenFeign performs inter-service REST calls; each microservice owns its own business data.